Void

Data.Void: https://hackage.haskell.org/package/base/docs/Data-Void.html

```
-- Data.Void
data Void
absurd :: Void -> a
```

Void
Type representing **non-existence**

Void	Void is non-existent	
a -> Void	a is non-existent by contradiction	i.e. if a exists then Void exists.
Void -> a	contradiction rule in first order logic	i.e. if Void exists, then anything exists

```
{-# OPTIONS_GHC  -Werror=incomplete-patterns #-}
{-# LANGUAGE LambdaCase #-}
{-# LANGUAGE EmptyCase #-}

import Data.Type.Equality

data Void
type Neg a = a -> Void

-- Void
isVoid :: Neg Void                          -- Void does not exist
isVoid void = \case void of {}
absurd :: Void -> a                         -- Contradiction if Void
exists
absurd void = case void of {}

-- e.g.
relationNot :: Neg (Int :~: Bool)           -- Int is not Bool
relationNot x = \case x of
relationAbsurd :: (Int :~: Bool) -> a       -- Contradiction if Int
is Bool
relationAbsurd x = case x of
```

==========

Dec

Data.Type.Dec: https://hackage.haskell.org/package/dec

```
-- Data.Type.Dec
data Dec a where
```

```
  Yes :: a -> Dec a
  No :: Neg a -> Dec a

type Neg a = a -> Void

class Decidable a where
  decide :: Dec a

toNegNeg :: a -> Neg (Neg a)
tripleNeg :: Neg (Neg (Neg a)) -> Neg a
contradict :: (a -> Neg b) -> b -> Neg a
contraposition :: (a -> b) -> Neg b -> Neg a
decNeg :: Dec a -> Dec (Neg a)
decShow :: Show a => Dec a -> String
decToMaybe :: Dec a -> Maybe a
decToBool :: Dec a -> Bool
boringYes :: Boring a => Dec a
absurdNo :: Absurd a => Dec a
```

Decidability
A type is **decidable** if a value (i.e. **witness**) can be **computed**.

v :: a	a value of a
v :: a -> Void	a non-existent value of a i.e. function should not be callable since value does not exist
x :: Dec a	
x = Yes (v :: a)	Decidable with witness v :: a
x = No (v :: a -> Void)	Non-decidable

e.g.

```
{-# OPTIONS_GHC  -Werror=incomplete-patterns #-}
{-# LANGUAGE LambdaCase #-}
{-# LANGUAGE EmptyCase #-}

import Data.Type.Dec
import Data.Type.Equality

intIsBool :: Neg (Int :~: Bool)
intIsBool x = case x of

intIsInt :: Int :~: Int
intIsInt = Refl

intIsIntN :: Neg (Neg (Int :~: Int))
intIsIntN nonRefl = nonRefl Refl            -- nonRefl :: Neg
(Int :~: Int) = Int :~: Int -> Void

-- e.g.
notIntIsBool y             -- IMPOSSIBLE. No such y :: (Int :~:
Bool)
intIsIntN y                -- IMPOSSIBLE. No such y :: Neg (Int :~:
Bool)
```

```
toNegNeg intIsInt          -- Neg (Neg (Int :~: Int))
```

Example: AND and OR witnesses

```
{-# LANGUAGE EmptyCase #-}
{-# LANGUAGE GADTs #-}
{-# LANGUAGE KindSignatures #-}
{-# LANGUAGE LambdaCase #-}
{-# LANGUAGE TypeOperators #-}
{-# OPTIONS_GHC -Werror=incomplete-patterns #-}

-- AND decision
data a :&&: b where
  (:&&:) :: a -> b -> a :&&: b

-- OR decisions
data a :||: b where
  MkL :: a -> a :||: b
  MkR :: b -> a :||: b

instance (Decidable a, Decidable b) => Decidable (a :&&: b) where
  decide = case decide @a of
             Yes a -> case decide @b of
                        Yes b -> Yes $ a :&&: b
                        No noB -> No $ \(_ :&&: b) -> noB b
             No noA -> No $ \(a :&&: _) -> noA a

instance (Decidable a, Decidable b) => Decidable (a :||: b) where
  decide = case decide @a of
             Yes a -> Yes $ MkL a
             No noA -> case decide @b of
                         Yes b -> Yes $ MkR b
                         No noB -> No $ \case
                          (OL a) -> noA a
                          (OR b) -> noB b



-- e.g.
data A = MkA
instance Decidable A where decide = Yes MkA             -- trivial
i.e. always exists

decide @(A :&&: A)               -- Yes (MkA :&&: MkA)
decide @(A :&&: Void)            -- No
decide @(Void :&&: A)            -- No
decide @(A :||: A)               -- Yes (MkL MkA)
decide @(A :||: Void)            -- Yes (MkL MkA)
decide @(Void :||: A)            -- Yes (MkR MkA)
decide @Void :||: Void)          -- No
```

==========

fin

fin: https://hackage.haskell.org/package/fin-0.2.1

fin
Type-level finite numbers

```
-- Data.Type.Nat
data Nat = Z | S Nat

data SNat (n :: Nat) where
 SZ :: SNat 'Z
 SS :: SNatI n => SNat ('S n)

class SNatI (n :: Nat) where
 induction :: f 'Z -> (forall m. SNatI m => f m -> f ('S m)) -> f n

snat :: SNatI n => SNat n
withSNat :: SNat n -> (SNatI n => r) -> r
reify :: forall r. Nat -> (forall n. SNatI n => r) -> r
reflect :: forall proxy m. SNatI m => proxy m -> Nat

discreteNat :: forall n m. (SNatI n, SNatI m) => Dec(n :~: m)
```

Usage: induction
e.g. build list of length n
buildNatList @Nat3 length 3 '['Nat3, 'Nat2, 'Nat1]

```
{-# LANGUAGE DataKinds #-}
{-# LANGUAGE ExistentialQuantification #-}
{-# LANGUAGE FlexibleInstances #-}
{-# LANGUAGE GADTs #-}
{-# LANGUAGE StandaloneKindSignatures #-}
{-# LANGUAGE TypeFamilies #-}
{-# LANGUAGE TypeOperators #-}
{-# LANGUAGE UndecidableInstances #-}

import Data.Kind
import Data.Proxy
import Data.Type.Nat

-- setup: type of the list
type NatList :: Nat -> [Nat]
type family NatList n where
  NatList 'Z = '[]
  NatList ('S n) = 'S n ': NatList n

-- setup: witness for building the list
type ListBuilder :: Nat -> Type
data ListBuilder n where
```

```
  MkListBuilder :: Proxy (NatList n) -> ListBuilder n

-- main: build list
buildNatList :: forall n. SNat n -> Proxy (NatList n)
buildNatList n = l
  where
    (MkListBuilder l) = res
    res :: ListBuilder n
    res = withSNat n $ induction (MkListBuilder (Proxy @'[])) step
    step :: forall m. ListBuilder m -> ListBuilder ('S m)
    step (MkListBuilder (_ :: Proxy l)) = MkListBuilder (Proxy @('S
m ': l))



-- e.g.
l0 :: Proxy (NatList 'Z)
l0 = buildNatList (snat @'Z)
l1 :: Proxy '[Nat1]
l1 = buildNatList (snat @Nat1)
l2 :: Proxy '[Nat2, Nat1]
l2 = buildNatList (snat @Nat2)
l3 :: Proxy '[Nat5, Nat4, Nat3, Nat2, Nat1]
l3 = buildNatList (snat @Nat5)
```

Example: Nat + LEProof implementation with Data.Singletons

Nat implementation

```
{-# LANGUAGE DataKinds #-}
{-# LANGUAGE EmptyCase #-}
{-# LANGUAGE FlexibleContexts #-}
{-# LANGUAGE GADTs #-}
{-# LANGUAGE ImpredicativeTypes #-}
{-# LANGUAGE InstanceSigs #-}
{-# LANGUAGE LambdaCase #-}
{-# LANGUAGE PatternSynonyms #-}
{-# LANGUAGE PolyKinds #-}
{-# LANGUAGE StandaloneKindSignatures #-}
{-# LANGUAGE TemplateHaskell #-}
{-# LANGUAGE TypeFamilies #-}
{-# LANGUAGE TypeOperators #-}
{-# LANGUAGE ViewPatterns #-}
{-# OPTIONS_GHC -Werror=incomplete-patterns #-}

import Control.Monad
import Data.Kind (Type)
import Data.Singletons.TH
import Data.Type.Equality (gcastWith)
import Data.Void
import GHC.Natural (Natural)
import Test.Hspec
```

```
data Nat = Z | S Nat deriving (Eq, Show)

type Neg a = a -> Void

genSingletons [''Nat]

instance Show (SNat n) where show = showSNat

type SomeNat = SomeSing Nat

pattern SomeNat :: SNat (n :: Nat) -> SomeSing Nat
pattern SomeNat n = SomeSing n

{-# COMPLETE SomeNat #-}

showSNat :: SNat n -> String
showSNat x = "SNat @" <> show (toNatural x)

toNatural :: SNat n -> Natural
toNatural SZ = 0
toNatural (SS x) = 1 + toNatural x
```

LEProof implementation

```
type LEProof :: Nat -> Nat -> Type
data LEProof m n where
  LEZero :: LEProof 'Z n -- witness: 0 <= x for any x
  LESucc :: LEProof m n -> LEProof ('S m) ('S n) -- witness: x <= y
implies x+1 <= y+1

data SomeLEProof where
  SomeLEProof :: forall m n. (SingI m, SingI n) => (LEProof m n) ->
SomeLEProof

instance Eq (LEProof x y) where _ == _ = True

instance (SingI x, SingI y) => Show (LEProof x y) where show =
showLEProof

showLEProof :: forall m n. (SingI m, SingI n) => LEProof m n ->
String
showLEProof _ = "LEProof: " <> showSNat (sing @m) <> " <= " <>
showSNat (sing @n)

showSomeLEProof :: SomeLEProof -> String
showSomeLEProof (SomeLEProof x) = showLEProof x

decideLE :: forall m n. (SingI m, SingI n) => Decision (LEProof m
n)
decideLE = decideLE' (sing @m) (sing @n)
```

```
decideLE' :: SNat m -> SNat n -> Decision (LEProof m n)
decideLE' SZ _ = Proved LEZero
decideLE' (SS m) (SS n) = case decideLE' m n of
  Proved proof -> Proved $ LESucc proof
  Disproved f -> Disproved $ \case
    LESucc proof -> f proof
decideLE' _ _ = Disproved $ \case {}

leToNat :: forall m n. (SingI m, SingI n) => LEProof m n -> (SNat
m, SNat n)
leToNat _ = (sing @m, sing @n)

leZero :: LEProof 'Z n
leZero = LEZero

leSucc :: LEProof m n -> LEProof ('S m) ('S n)
leSucc = LESucc

leRefl :: forall n. SingI n => LEProof n n
leRefl = case sing @n of
  SZ -> LEZero
  SS x -> withSingI x leRefl

lePred :: LEProof ('S m) ('S n) -> LEProof m n
lePred (LESucc proof) = proof

leStep :: LEProof m n -> LEProof m ('S n)
leStep LEZero = LEZero
leStep (LESucc proof) = LESucc $ leStep proof

leStepL :: LEProof ('S m) n -> LEProof m n
leStepL x = let LESucc y = leStep x in y

leAsym :: LEProof m n -> LEProof n m -> n :~: m
leAsym LEZero LEZero = Refl
leAsym (LESucc m_) (LESucc n_) = gcastWith (leAsym m_ n_) Refl

leTrans :: LEProof m n -> LEProof n o -> LEProof m o
leTrans LEZero _ = LEZero
leTrans (LESucc m_) (LESucc n_) = leSucc $ leTrans m_ n_

proofZeroLEZero :: LEProof n 'Z -> n :~: 'Z
proofZeroLEZero LEZero = Refl

leSwap :: SNat m -> SNat n -> Neg (LEProof m n) -> LEProof ('S n) m
leSwap (SS m_) (SS n_) f = LESucc $ leSwap m_ n_ (f . LESucc)
leSwap SZ _ f = absurd $ f LEZero
leSwap (SS _) SZ _ = LESucc LEZero

leSwap' :: LEProof n m -> LEProof ('S m) n -> Void
leSwap' = ...?
```

==========

OLD